

SpecFirst Phase 1 Architecture

React Component Analysis with ts-morph

Executive Summary

Phase 1 is the static-analysis layer of the SpecFirst pipeline. It parses a React TypeScript component with ts-morph, extracts a normalized component-facts model, and writes componentAnalysis.json. The phase collects evidence only. It does not classify the component pattern, load accessibility rules, generate tests, patch code, or make WCAG claims.

1. Purpose of Phase 1

Phase 1 answers one question: what is physically present inside this React component file? Its output is a reliable, machine-readable artifact that later phases can consume without rereading source code.

The analyzer should extract enough facts to support later pattern classification, rule selection, Bob patch constraints, and the final evidence report.

- Component identity: name, file path, export style, declaration type.
- Public API: props, TypeScript types, callbacks, labels, values, collections.
- State model: useState hooks and likely state purpose such as open/close or selection.
- JSX model: native elements, custom components, attributes, event handlers, ARIA attributes.
- Render patterns: conditional popup, options rendered with map, trigger controlling popup state.
- Risk flags: static-analysis warnings such as click behavior without keyboard behavior.

2. Why ts-morph Instead of Regex

Regex is acceptable for a throwaway prototype, but it is too shallow for this product. SpecFirst needs structural knowledge of React and TypeScript. ts-morph gives the team a higher-level interface over the TypeScript compiler API so the analyzer can traverse actual AST nodes rather than raw strings.

Need	Regex	ts-morph
Find exported React components	Brittle	Strong
Extract prop types	Poor	Strong
Inspect JSX tree	Weak	Strong
Detect state hooks	Partial	Strong
Analyze event handlers	Shallow	Good
Detect ARIA attributes	Basic	Good
Support future code modifications	No	Possible

Recommended choice: use ts-morph for Phase 1. Keep Babel or direct TypeScript Compiler API as alternatives, but do not make regex the primary analyzer.

3. Engineering Boundary

Phase 1 should do	Phase 1 should not do
Parse a React .tsx/.jsx file	Make final component-pattern decisions
Find target component declaration	Load accessibility manifests
Extract props, state, JSX, event handlers, ARIA signals	Generate WCAG/ARIA requirements
Detect render patterns	Generate Playwright tests
Generate static risk flags	Call Bob to patch code
Write componentAnalysis.json	Claim the component violates or satisfies WCAG

The clean architectural boundary is: Phase 1 collects facts, Phase 2 classifies pattern, Phase 3 loads rules, Phase 4 freezes a component-specific contract.

4. Input and Output

Input

A single target component path:

```
npm run specfirst:analyze -- src/components/FruitDropdown.tsx
```

Future optional inputs can include a target export name, project root, tsconfig path, demo route, or component hint. For the MVP, component path alone should be enough.

Output

Phase 1 writes a deterministic artifact:

```
specfirst/runs/<run-id>/componentAnalysis.json
```

This artifact is consumed directly by Phase 2. Later phases should not have to re-analyze the component source to understand the basic component structure.

5. Recommended File Structure

```
specfirst/  
  core/  
    analysis/  
      analyzeComponent.ts  
      findComponents.ts  
      extractProps.ts  
      extractStateHooks.ts  
      extractJsxTree.ts  
      extractEventHandlers.ts  
      extractA11ySignals.ts  
      detectRenderPatterns.ts  
      generateRiskFlags.ts  
      types.ts  
  
  cli/  
    analyze.ts  
  
  runs/  
    001/  
      componentAnalysis.json
```

This modular structure lets multiple engineers work in parallel. Avoid putting the whole analyzer into one giant file.

6. Phase 1 Data Model

The analyzer should emit a rich object, not a flat collection of booleans. The proposed schema is below.

```
type ComponentAnalysis = {  
  schemaVersion: "1.0.0"  
  
  component: {  
    name: string  
    path: string  
    framework: "react"  
    language: "typescript" | "javascript"  
    declarationKind: "function" | "arrow-function" | "unknown"  
    exportType: "named" | "default" | "unknown"  
  }  
  
  props: Array<{  
    name: string  
    type?: string  
    required?: boolean  
    kind?: "value" | "callback" | "collection" | "label" | "boolean" | "styling" | "unknown"  
  }>  
  
  state: Array<{  
    name: string  
    setter: string  
    initialValue?: string  
    likelyPurpose?: "open-close" | "selection" | "active-index" | "unknown"  
  }>  
  
  jsxElements: Array<{  
    id: string  
    tag: string  
    isNativeElement: boolean
```

```

    attributes: string[]
    eventHandlers: string[]
    ariaAttributes: string[]
    roleValue?: string
    insideMap?: boolean
    insideConditional?: boolean
    textHints?: string[]
  }>

  renderPatterns: {
    conditionalPopup: boolean
    popupControlledByState?: string
    optionsRenderedFromMap: boolean
    triggerControlsPopup: boolean
    selectableOptions: boolean
  }

  accessibilitySignals: {
    roles: string[]
    ariaAttributes: string[]
    hasAccessibleNameCandidate: boolean
    hasKeyboardHandler: boolean
    hasComboboxRole: boolean
    hasListboxRole: boolean
    hasOptionRole: boolean
    hasAriaExpanded: boolean
    hasAriaControls: boolean
    hasAriaSelected: boolean
    hasAriaActiveDescendant: boolean
  }

  eventStateLinks: Array<{
    elementId: string
    event: string
    callsSetter?: string
    state?: string
    likelyAction?: "toggle" | "set" | "select" | "unknown"
  }>

  riskFlags: Array<{
    id: string
    severity: "low" | "medium" | "high"
    message: string
  }>

  limitations: string[]
}

```

7. Analyzer Passes

Phase 1 should run as a pipeline of analysis passes. Each pass should be independently testable.

1. Initialize ts-morph project and load SourceFile.
2. Find target React component declaration.
3. Extract props and TypeScript types.
4. Extract state hooks.
5. Extract JSX elements.
6. Extract event handlers and event-to-state links.
7. Extract roles and ARIA attributes.

8. Detect render patterns.
9. Generate static accessibility risk flags.
10. Write componentAnalysis.json and terminal summary.

8. Pass 1: Initialize ts-morph Project

The analyzer should prefer the project tsconfig so TypeScript types resolve correctly.

```
import { Project } from "ts-morph"

const project = new Project({
  tsConfigFilePath: "tsconfig.json",
  skipAddingFilesFromTsConfig: false
})

const sourceFile = project.getSourceFileOrThrow(targetPath)
```

If no tsconfig is available, fall back to directly adding the file. For the MVP, a standard Vite React TypeScript app is enough.

```
const project = new Project({})
const sourceFile = project.addSourceFileAtPath(targetPath)
```

9. Pass 2: Find Target Component

Support the most common React declaration styles: named function exports, named arrow function exports, and default exports.

```
export function FruitDropdown() { return <div /> }

export const FruitDropdown = () => { return <div /> }

function FruitDropdown() { return <div /> }
export default FruitDropdown
```

Selection priority should be: explicit target export if provided, exported component with capitalized name, then first capitalized function/arrow component in the file. If multiple likely components exist and no export is clear, emit an ambiguity warning.

10. Pass 3: Extract Props and Types

Props are important because Bob must preserve the component public API during remediation.

```
type FruitDropdownProps = {
  label: string
  options: Option[]
  value?: string
  onChange: (value: string) => void
  disabled?: boolean
  className?: string
}

export function FruitDropdown({ label, options, value, onChange }: FruitDropdownProps) {
  // ...
}
```

Expected extracted prop records include name, type, required/optional status, and inferred kind.

Prop	Type	Required	Kind
------	------	----------	------

label	string	yes	label
options	Option[]	yes	collection
value	string undefined	no	value
onChange	(value: string) => void	yes	callback
disabled	boolean undefined	no	boolean
className	string undefined	no	styling

11. Pass 4: Extract State Hooks

Detect React state hooks, especially state that controls visibility, active index, or selected value.

```
const [open, setOpen] = useState(false)
const [activeIndex, setActiveIndex] = useState(0)
```

State name	Setter	Initial value	Likely purpose
open	setOpen	false	open-close
activeIndex	setActiveIndex	0	active-index

Heuristics: open/isOpen/expanded/visible imply open-close; selected/value imply selection; activeIndex/highlightedIndex imply active-index.

12. Pass 5: Extract JSX Elements

Build a normalized inventory of rendered JSX elements. Track tag, attributes, event handlers, ARIA attributes, whether it appears inside a map, and whether it appears inside conditional rendering.

```
<button onClick={() => setOpen(!open)}>
  {selected?.label ?? "Choose fruit"}
</button>

{open && (
  <ul className="dropdown-list">
    {options.map((option) => (
      <li key={option.value} onClick={() => onChange(option.value)}>
        {option.label}
      </li>
    ))}
  </ul>
)}
```

Element	Attributes	Event handlers	Context
button	onClick	onClick	outside map, outside conditional
ul	className	-	inside conditional
li	key, onClick	onClick	inside map and conditional

Lowercase tags are native elements. Capitalized tags are custom components and should be recorded but not resolved cross-file in the MVP.

13. Pass 6: Extract Event Handlers

Detect interactive behavior attached to JSX elements. Relevant events include `onClick`, `onKeyDown`, `onKeyUp`, `onFocus`, `onBlur`, `onChange`, and `onMouseDown`.

```
<button onClick={() => setOpen(!open)}>
  ...
should produce:
...
{
  "elementId": "jsx-1",
  "event": "onClick",
  "callsSetter": "setOpen",
  "state": "open",
  "likelyAction": "toggle"
}
```

This event-to-state link is high-value because it shows that the button likely controls the popup rendered by the same open state.

14. Pass 7: Extract Accessibility Signals

Extract existing roles and ARIA attributes. Relevant attributes include `role`, `aria-expanded`, `aria-controls`, `aria-selected`, `aria-activedescendant`, `aria-labelledby`, `aria-label`, and `tabIndex`.

```
<button
  role="combobox"
  aria-expanded={open}
  aria-controls="fruit-listbox"
>
```

Expected accessibility signal output:

```
{
  "roles": ["combobox"],
  "ariaAttributes": ["aria-expanded", "aria-controls"],
  "hasAccessibleNameCandidate": true,
  "hasKeyboardHandler": false,
  "hasComboboxRole": true,
  "hasListboxRole": false,
  "hasOptionRole": false,
  "hasAriaExpanded": true,
  "hasAriaControls": true,
  "hasAriaSelected": false,
  "hasAriaActiveDescendant": false
}
```

Do not implement the full accessible-name algorithm in Phase 1. Detect candidates only: visible trigger text, `aria-label`, `aria-labelledby`, `label` prop, or nearby label element.

15. Pass 8: Detect Render Patterns

This pass summarizes structural relationships that are useful for classification.

```
{
  "conditionalPopup": true,
  "popupControlledByState": "open",
  "optionsRenderedFromMap": true,
  "triggerControlsPopup": true,
}
```

```
"selectableOptions": true
}
```

Pattern	Detection rule
conditionalPopup	JSX conditionally renders popup-like element such as ul/div/List
popupControlledByState	Conditional expression references open-like state variable
optionsRenderedFromMap	JSX contains .map() that outputs option-like nodes
triggerControlsPopup	Trigger event handler calls setter for state controlling popup
selectableOptions	Mapped items have onClick/onSelect or call onChange

16. Pass 9: Generate Risk Flags

Risk flags are static-analysis warnings. They are not final accessibility judgments.

Risk ID	Severity	Trigger
click-without-keyboard	high	Click handler exists on trigger but no keyboard handler
missing-expanded-state	medium	Conditional popup exists but no aria-expanded
missing-popup-association	medium	Trigger/popup relationship exists but no aria-controls
missing-option-semantics	medium	Mapped selectable items lack role="option"
missing-selected-state	medium	Selectable list lacks aria-selected
custom-component-unresolved	low/medium	Interaction happens inside custom component not resolved in MVP
possible-pattern-ambiguity	medium	Signals could indicate multiple UI patterns

17. Example componentAnalysis.json

```
{
  "schemaVersion": "1.0.0",
  "component": {
    "name": "FruitDropdown",
    "path": "src/components/FruitDropdown.tsx",
    "framework": "react",
    "language": "typescript",
    "declarationKind": "function",
    "exportType": "named"
  },
  "props": [
    { "name": "label", "type": "string", "required": true, "kind": "label" },
    { "name": "options", "type": "Option[]", "required": true, "kind": "collection" },
    { "name": "value", "type": "string | undefined", "required": false, "kind": "value" },
    { "name": "onChange", "type": "(value: string) => void", "required": true, "kind": "callback" }
  ]
}
```



```

],
"state": [
  { "name": "open", "setter": "setOpen", "initialValue": "false", "likelyPurpose": "open-close" }
],
"jsxElements": [
  {
    "id": "jsx-1",
    "tag": "button",
    "isNativeElement": true,
    "attributes": ["className", "onClick"],
    "eventHandlers": ["onClick"],
    "ariaAttributes": [],
    "insideMap": false,
    "insideConditional": false
  },
  {
    "id": "jsx-2",
    "tag": "ul",
    "isNativeElement": true,
    "attributes": ["className"],
    "eventHandlers": [],
    "ariaAttributes": [],
    "insideMap": false,
    "insideConditional": true
  },
  {
    "id": "jsx-3",
    "tag": "li",
    "isNativeElement": true,
    "attributes": ["key", "className", "onClick"],
    "eventHandlers": ["onClick"],
    "ariaAttributes": [],
    "insideMap": true,
    "insideConditional": true
  }
],
"renderPatterns": {
  "conditionalPopup": true,
  "popupControlledByState": "open",
  "optionsRenderedFromMap": true,
  "triggerControlsPopup": true,
  "selectableOptions": true
},
"accessibilitySignals": {
  "roles": [],
  "ariaAttributes": [],
  "hasAccessibleNameCandidate": true,
  "hasKeyboardHandler": false,
  "hasComboboxRole": false,
  "hasListboxRole": false,
  "hasOptionRole": false,
  "hasAriaExpanded": false,
  "hasAriaControls": false,
  "hasAriaSelected": false,
  "hasAriaActiveDescendant": false
},
"eventStateLinks": [
  {
    "elementId": "jsx-1",
    "event": "onClick",
    "callsSetter": "setOpen",
    "state": "open",

```

```

    "likelyAction": "toggle"
  }
],
"riskFlags": [
  {
    "id": "click-without-keyboard",
    "severity": "high",
    "message": "Trigger has click behavior but no keyboard handler."
  },
  {
    "id": "missing-expanded-state",
    "severity": "medium",
    "message": "Popup visibility state exists but is not exposed through aria-expanded."
  },
  {
    "id": "missing-option-semantic",
    "severity": "medium",
    "message": "Mapped selectable options do not expose option semantics."
  }
],
"limitations": [
  "Imported custom components are not resolved in this MVP.",
  "Screen reader behavior is not inferred from static analysis.",
  "Accessible name candidates are detected, not fully computed."
]
}

```

18. Implementation Sketch

```

import { Project } from "ts-morph"
import { findTargetComponent } from "./findComponents"
import { extractProps } from "./extractProps"
import { extractStateHooks } from "./extractStateHooks"
import { extractJsxTree } from "./extractJsxTree"
import { extractA11ySignals } from "./extractA11ySignals"
import { detectRenderPatterns } from "./detectRenderPatterns"
import { generateRiskFlags } from "./generateRiskFlags"
import type { ComponentAnalysis } from "./types"

export function analyzeComponent(targetPath: string): ComponentAnalysis {
  const project = new Project({
    tsConfigFilePath: "tsconfig.json",
    skipAddingFilesFromTsConfig: false
  })

  const sourceFile = project.getSourceFileOrThrow(targetPath)
  const component = findTargetComponent(sourceFile)

  const props = extractProps(component.node)
  const state = extractStateHooks(component.node)
  const jsxElements = extractJsxTree(component.node)
  const accessibilitySignals = extractA11ySignals(jsxElements)
  const renderPatterns = detectRenderPatterns(component.node, jsxElements, state)
  const riskFlags = generateRiskFlags({ jsxElements, accessibilitySignals, renderPatterns, state })

  return {
    schemaVersion: "1.0.0",
    component: {
      name: component.name,
      path: targetPath,
      framework: "react",

```

```

    language: targetPath.endsWith(".tsx") ? "typescript" : "javascript",
    declarationKind: component.declarationKind,
    exportType: component.exportType
  },
  props,
  state,
  jsxElements,
  renderPatterns,
  accessibilitySignals,
  eventStateLinks: renderPatterns.eventStateLinks ?? [],
  riskFlags,
  limitations: [
    "Imported custom components are not resolved in the MVP.",
    "Screen reader behavior is not inferred from static analysis.",
    "Accessible name candidates are detected, not fully computed."
  ]
}
}

```

19. CLI Sketch

```

import fs from "fs"
import path from "path"
import { analyzeComponent } from "../core/analysis/analyzeComponent"

const targetPath = process.argv[2]

if (!targetPath) {
  console.error("Usage: npm run specfirst:analyze -- <component-path>")
  process.exit(1)
}

if (!fs.existsSync(targetPath)) {
  console.error(`File not found: ${targetPath}`)
  process.exit(1)
}

const analysis = analyzeComponent(targetPath)

const runDir = "specfirst/runs/001"
fs.mkdirSync(runDir, { recursive: true })

const outputPath = path.join(runDir, "componentAnalysis.json")
fs.writeFileSync(outputPath, JSON.stringify(analysis, null, 2))

console.log("SpecFirst Phase 1: Component Analysis")
console.log(`Component: ${analysis.component.name}`)
console.log(`Path: ${analysis.component.path}`)
console.log(`Output: ${outputPath}`)

if (analysis.riskFlags.length > 0) {
  console.log("Risk flags:")
  for (const flag of analysis.riskFlags) {
    console.log(`- ${flag.severity.toUpperCase()}: ${flag.message}`)
  }
}

{
  "scripts": {
    "specfirst:analyze": "tsx specfirst/cli/analyze.ts"
  }
}

```

20. MVP vs Advanced Scope

MVP scope	Defer until after MVP
Component discovery	Cross-file import resolution
Prop extraction	Design-system primitive resolution
State hook extraction	Full accessible-name computation
JSX element extraction	Full control-flow/data-flow analysis
Event handler extraction	Automatic Storybook story discovery
ARIA/role extraction	Runtime rendering analysis
Conditional and map detection	Screen reader behavior simulation
Risk flags and JSON output	Automated remediation in Phase 1

21. Failure Modes and Handling

Failure	Example	Handling
Multiple components in one file	Dropdown, Option, Trigger	Choose exported component or require target export
Custom component used	<code><Button onClick={...}></code>	Record unresolved custom component limitation
Props cannot be resolved	Untyped JavaScript component	Extract names from destructuring if possible
State hook not obvious	Reducer or external store	Record limitation
Conditional popup not detected	Complex helper function	Lower Phase 2 confidence
ARIA spread object	<code>{...ariaProps}</code>	Record spread attributes and uncertainty
No JSX found	Component delegates render elsewhere	Mark unsupported for MVP

22. Definition of Done for Phase 1

- CLI accepts a target React component path.
- ts-morph project initializes successfully.
- Target component is found.
- Props and prop types are extracted when available.
- State hooks are extracted.
- JSX elements are extracted.
- Event handlers are extracted.
- ARIA and role attributes are extracted.
- Conditional rendering is detected.
- Mapped option rendering is detected.
- Event-to-state links are detected for simple handlers.
- Risk flags are generated.

- componentAnalysis.json is written.
- Terminal summary is readable.
- Limitations are explicitly recorded.

23. Final Architectural Summary

Phase 1 should be a static analysis engine, not a regex scanner and not an AI guesser.

```
ts-morph parser
↓
component discovery
↓
props/state extraction
↓
JSX/event/a11y extraction
↓
render-pattern detection
↓
risk flag generation
↓
componentAnalysis.json
```

This gives SpecFirst a serious foundation. Phase 2 can classify the component based on real code structure rather than vague text matching. Phase 7 can instruct Bob to preserve the actual public API. Phase 8 can include analysis evidence in the final report.

The main engineering rule: Phase 1 collects facts. It does not make final accessibility claims.